Air University

Cryptography (Lab)

Year 2026

Mr. Omer Ali

Academic Year: 2024 – 2028

Department of Cyber Security

USAMA ARSHAD | 247141

BS – Cyber Security – Fall – 24 (A)

Date of Submission: May 14, 2026

Lab # 12

Implementation of RSA and DSA using OpenSSL

Environment Preparation: The mkdir command creates a dedicated directory named Lab_12, and the openssl version confirms that OpenSSL 3.5.5 is installed. The cd command is then used to move into the newly created workspace.

```
┌──(kali㉿kali)-[~/Documents]
└─$ mkdir Lab_12

┌──(kali㉿kali)-[~/Documents]
└─$ openssl version
OpenSSL 3.5.5 27 Jan 2026 (Library: OpenSSL 3.5.5 27 Jan 2026)

┌──(kali㉿kali)-[~/Documents]
└─$ cd Lab_12
```

Part A: RSA Implementation

1. Generate RSA Private Key

The openssl genpkey command generates a 2048-bit RSA private key saved as rsa_private.pem. The ls -la command verifies the file was created successfully with the correct system permissions.

```
┌──(kali㉿kali)-[~/Documents/Lab_12]
└─$ openssl genpkey -algorithm RSA -out rsa_private.pem -pkeyopt rsa_keygen_bits:2048
.+..+.........+.....+.......+..............+..+.................+..++++++++++++++++++++++++++++++++++++++++*......+.+..+...+....+..+..........+.....+....+.....+......+.+..+....
..+..+.............+..+.............+...+..+...+.............+............+.....+....+......+.....+...+....+..+......+...+....+................+..+...+.......+........+.+......+...
++++++++++++++++++++++...+++++++
....+....+......+..+.......+.........+.....+.+.....+.+..+.............+...++++++++++++++++++++++++++++++++++++++++*..........+....+..+.++++++++++++++++++++++++++++++++++++++++*....
+..+.+...+.........+...+.....+....+........+......+.......+...++++++

┌──(kali㉿kali)-[~/Documents/Lab_12]
└─$ ls -la rsa_private.pem
-rw------- 1 kali kali 1704 May  7 13:24 rsa_private.pem
```

2. Extract Public Key

The openssl rsa utility extracts the public key from the private key file and saves it as rsa_public.pem. Listing the files confirms both halves of the asymmetric key pair are now present in the directory.

```
┌──(kali㉿kali)-[~/Documents/Lab_12]
└─$ openssl rsa -pubout -in rsa_private.pem -out rsa_public.pem
writing RSA key

┌──(kali㉿kali)-[~/Documents/Lab_12]
└─$ ls -la rsa_*.pem
-rw------- 1 kali kali 1704 May  7 13:24 rsa_private.pem
-rw-rw-r-- 1 kali kali  451 May  7 13:27 rsa_public.pem
```

3. Create a Plaintext Message File

A text file named secret_msg.txt is created using the nano editor to store the original message. The cat command displays the file content to verify the plaintext before encryption.

```
┌──(kali㉿kali)-[~/Documents/Lab_12]
└─$ nano secret_msg.txt

┌──(kali㉿kali)-[~/Documents/Lab_12]
└─$ cat secret_msg.txt
this is lab 12. This is secret text for RSA.
```

4. Encrypt using the Public Key

The message is encrypted using the recipient's public key with the pkeyutl command to produce message_1.enc. The xxd and hexdump tools display the resulting ciphertext in hexadecimal format, showing it is unreadable.

```
┌──(kali㉿kali)-[~/Documents/Lab_12]
└─$ openssl pkeyutl -encrypt -in secret_msg.txt -out message_1.enc -pubin -inkey rsa_public.pem

┌──(kali㉿kali)-[~/Documents/Lab_12]
└─$ xxd message_1.enc
00000000: 03c8 17ee 6d67 361c 7d64 9ae8 2817 01c4  ....mg6.}d..( ...
00000010: 076f c578 5a2d 1f94 8364 fba7 33a4 42b0  .o.xZ-...d..3.B.
00000020: c80c 9b82 dbef 3030 0f5c 40ea 12ba 7f6c  ......00.\@....l
00000030: 6f8e 1723 f819 69fa 04cd 7ed5 6b77 1776  o..#..i...~.kw.v
00000040: 92fe 2119 879e adcc 5eb1 4503 aaab 2a73  ..!.....^.E...*s
00000050: a6c0 8db1 06a0 fb3d a21b 6d52 25fe d5ab  .......=..mR%...
00000060: f70e bf80 c48b 555b a729 3db9 d9e5 682c  ......U[.)=...h,
00000070: 711d a685 13e7 ec0e c7a8 7a57 e821 f079  q.........zW.!.y
00000080: 5703 0f93 9f66 88ca e2d3 7cab 7690 a819  W....f....|.v...
00000090: 49db 6088 981d 01ad c18f 6831 bdca 0357  I.`.......h1...W
000000a0: 000e 90e1 c45c dbeb 35a8 7604 2408 1809  .....\..5.v.$...
000000b0: 6793 2995 6105 1d70 e98b 5796 4e83 2a9a  g.).a..p..W.N.*.
000000c0: b4ec 7eba 4c71 30ee 1b34 43cf 25e3 806b  ..~.Lq0..4C.%..k
000000d0: fa39 dc47 9756 ca76 4acc a8ed 9faf cfc6  .9.G.V.vJ.......
000000e0: 988f 177a d1f5 fd0f 42d7 87c0 bf22 62ef  ...z....B....."b.
000000f0: ad7a 9287 7dca 52b7 d876 15c5 ddf8 145c  .z..}.R..v.....\

┌──(kali㉿kali)-[~/Documents/Lab_12]
└─$ hexdump -C message_1.enc
00000000  03 c8 17 ee 6d 67 36 1c  7d 64 9a e8 28 17 01 c4  |....mg6.}d..( ...|
00000010  07 6f c5 78 5a 2d 1f 94  83 64 fb a7 33 a4 42 b0  |.o.xZ-...d..3.B.|
00000020  c8 0c 9b 82 db ef 30 30  0f 5c 40 ea 12 ba 7f 6c  |......00.\@....l|
00000030  6f 8e 17 23 f8 19 69 fa  04 cd 7e d5 6b 77 17 76  |o..#..i...~.kw.v|
00000040  92 fe 21 19 87 9e ad cc  5e b1 45 03 aa ab 2a 73  |..!.....^.E...*s|
00000050  a6 c0 8d b1 06 a0 fb 3d  a2 1b 6d 52 25 fe d5 ab  |.......=..mR%...|
00000060  f7 0e bf 80 c4 8b 55 5b  a7 29 3d b9 d9 e5 68 2c  |......U[.)=...h,|
00000070  71 1d a6 85 13 e7 ec 0e  c7 a8 7a 57 e8 21 f0 79  |q.........zW.!.y|
00000080  57 03 0f 93 9f 66 88 ca  e2 d3 7c ab 76 90 a8 19  |W....f....|.v...|
00000090  49 db 60 88 98 1d 01 ad  c1 8f 68 31 bd ca 03 57  |I.`.......h1...W|
000000a0  00 0e 90 e1 c4 5c db eb  35 a8 76 04 24 08 18 09  |.....\..5.v.$...|
000000b0  67 93 29 95 61 05 1d 70  e9 8b 57 96 4e 83 2a 9a  |g.).a..p..W.N.*.|
000000c0  b4 ec 7e ba 4c 71 30 ee  1b 34 43 cf 25 e3 80 6b  |..~.Lq0..4C.%..k|
000000d0  fa 39 dc 47 97 56 ca 76  4a cc a8 ed 9f af cf c6  |.9.G.V.vJ.......|
000000e0  98 8f 17 7a d1 f5 fd 0f  42 d7 87 c0 bf 22 62 ef  |...z....B...."b.|
000000f0  ad 7a 92 87 7d ca 52 b7  d8 76 15 c5 dd f8 14 5c  |.z..}.R..v.....\|
00000100
```

5. Decrypt Using the Private Key

The pkeyutl command uses the corresponding RSA private key to decrypt the ciphertext back into a readable text file. The cat command demonstrates that the decrypted output matches the original secret message exactly.

```
┌──(kali㉿kali)-[~/Documents/Lab_12]
└─$ openssl pkeyutl -decrypt -in message_1.enc -out msg_decrypted_1.txt -inkey rsa_private.pem

┌──(kali㉿kali)-[~/Documents/Lab_12]
└─$ cat msg_decrypted_1.txt
this is lab 12. This is secret text for RSA.
```

6. Generate a Digital Signature

A digital signature is created by hashing the message with SHA-256 and signing it with the private key using openssl dgst. The xxd tool displays the signature's binary data, which provides authenticity and non-repudiation.

```
┌──(kali㉿kali)-[~/Documents/Lab_12]
└─$ openssl dgst -sha256 -sign rsa_private.pem -out signature_1.bin secret_msg.txt

┌──(kali㉿kali)-[~/Documents/Lab_12]
└─$ ls -la signature_1.bin
-rw-rw-r-- 1 kali kali 256 May  7 13:39 signature_1.bin

┌──(kali㉿kali)-[~/Documents/Lab_12]
└─$ xxd signature_1.bin
00000000: 6d70 5ba6 717f 5a47 565d 4647 dfc8 4a6c  mp[.q.ZGV]FG..Jl
00000010: 8aa2 1031 f821 900c 5d68 7d54 f503 c570  ...1.!..]h}T...p
00000020: 18ed 1b6a 0e54 cdfa e30c ffd3 e1af b587  ...j.T..........
00000030: bf4a 041b 6413 f4fb 307d 61e8 2047 4fcf  .J..d...0}a. GO.
00000040: 5c6b f22e 47e7 a8c0 89ec f589 fc14 7d6d  \k..G.........}m
00000050: 539a 9e21 26b0 5208 6ca3 c07f 1afd 447a  S..!&.R.l.....Dz
00000060: 6344 2c87 478e b957 1bd3 f308 f225 6ae4  cD,.G..W.....%j.
00000070: 7d7e 3977 befa 71b0 4052 0cbd 9208 8f15  }~9w..q.@R......
00000080: d0dc 5ac1 e92f 405e 71f1 6018 6e40 d460  ..Z../@^q.`.n@.`
00000090: 5b31 8614 a4bf 7760 81d2 84ea 0fb0 08e1  [1....w`........
000000a0: 55fb e3c8 4afb 5785 8e20 2299 dc15 199e  U...J.W.. ".....
000000b0: cb45 2ecb e44a 41e2 fb0e 5703 1d4f a996  .E...JA...W..O..
000000c0: 3e52 f650 944e 7d4d 7ad2 d9fd ccce 65bc  >R.P.N}Mz.....e.
000000d0: 376c 1f43 e7b1 279a 9ece 79d1 c950 848d  7l.C..'...y..P..
000000e0: fd72 d01e 9f08 6efd 7053 d576 c941 1b2e  .r....n.pS.v.A..
000000f0: 6f33 298a 93df d3cd 42c5 5103 ae9b 5973  o3).....B.Q...Ys
```

7. Verify the Digital Signature

The openssl dgst -verify command uses the public key to check the signature against the original message file. The "Verified OK" output confirms that the message has not been altered and was indeed signed by the private key holder.

Observation

- **Asymmetric Transformation:** The hex dump and `xxd` output showed that the encryption process transformed the plaintext into an unreadable hexadecimal format, making it secure without the corresponding key.
- **File Integrity:** The successful decryption and verification confirmed that the data remained intact throughout the cryptographic cycle.
- **Signature Size:** The signing process yielded a 256-byte signature file, which acts as a cryptographic thumbprint linking it to the specific source file and private key used.

Result

- **Key Generation:** A 2048-bit RSA private key (`***rsa_private.pem***`) was successfully generated, and the corresponding public key (`***rsa_public.pem***`) was extracted for use in asymmetric operations.
- **Encryption & Decryption:** A plaintext message, "This is lab 12. This is secret text for RSA," was saved in `***secret_msg.txt***` and encrypted using the public key. The resulting ciphertext was then decrypted back to its original form using the private key.
- **Signature Verification:** A digital signature (`***signature_1.bin***`) was created by signing the message with the private key. Verification against the public key confirmed the process with a "**Verified OK**."

Conclusion

The RSA implementation demonstrates that this algorithm can effectively deliver both data confidentiality and authenticity. By leveraging a linked public-private key pair, the system ensures that only the intended recipient has access to sensitive information, while also providing a means to verify the sender's identity and the integrity of the message. The "Verified OK" outcome reinforces the operational integrity of the asymmetric cryptographic chain as a reliable security measure for secure communications.

- End of Lab Task -